

# Projet Final — Agence Marketing Data & API Intelligence avec ML

2–3 séances (≈7–10h) · Binôme/Trinôme

## Objectif

Concevoir un outil qui :

1. **Collecte** des données via plusieurs APIs publiques
2. **Nettoie/structure** les contenus textuels
3. **Analyse & visualise** des indicateurs clés
4. **Applique du ML** (Scikit-learn) pour classifier/segmenter ou recommander des contenus
5. **Produit un rapport** synthétique + figures

## Arborescence & rôles (ML inclus)

```
/marketing_ai/
├─ main.py                # orchestration complète (ETL + Viz + ML +
rapport)
├─ core/
│  └─ config.py           # URLs, tokens, params ML (seed,
test_size, top_k)
│  └─ fetcher.py          # appels API, statuts, latences
│  └─ cleaner.py          # normalisation texte (minuscule,
punctuation, stopwords FR/EN)
│  └─ analyzer.py         # KPIs descriptifs (longueurs, fréquences,
sources)
│  └─ features.py         # extraction features (TF-IDF, n-grams,
options)
│  └─ model.py            # entraînement/éval (clf ou clustering),
sauvegarde .pkl
│  └─ recommender.py      # similarité cosinus top-k (option "reco")
│  └─ viz.py              # figures obligatoires (PNG/SVG) +
dashboard.pdf
├─ data/
│  └─ raw/                # dumps bruts
│  └─ processed/          # clean_data.json / features.npz
│  └─ models/             # model.pkl / vectorizer.pkl
├─ reports/
│  └─ summary.json
│  └─ keywords.csv
│  └─ dashboard.pdf
├─ figs/
│  └─ sources_bar.png
│  └─ top_keywords.png
│  └─ latency_box.png
│  └─ status_codes.png
│  └─ timeline_activity.png
└─ logs/
   └─ marketing.log
```

## Exigences techniques (ML)

- **Jeu de données** : contenu textuel issu de  $\geq 3$  **APIs** (ex : actus/produits/tendances).
- **Pré-traitement texte** minimal : minuscule, suppression ponctuation, stopwords (FR/EN), éventuellement lemmatisation si disponible.
- **Vectorisation** : `TfidfVectorizer` (ngrams paramétrables, ex : 1–2).
- **Tâche ML** (choisir **AU MOINS 1** scénario) :
  1. **Classification** supervisée (si labels disponibles via API) : `LogisticRegression` / `LinearSVC`.
  2. **Clustering** non supervisé : `KMeans` (k justifié).
  3. **Recommandation** basique : similarité cosinus (top-k documents proches d’une requête).
- **Évaluation** :
  - Classification : **train/test split** (seed fixé), métriques **accuracy** + **F1** (macro si classes déséquilibrées).
  - Clustering : **silhouette score** + interprétation (top n-grams par cluster).
  - Reco : **sanity check** (exemples concrets + justification des résultats).
- **Reproductibilité** : `random_state` fixé, `requirements.txt`, sauvegarde `model.pkl` & `vectorizer.pkl`.
- **Sans deep learning ; Scikit-learn uniquement.**

## Data Visualisation (obligatoire)

- Minimum **5 figures** (PNG/SVG) + **dashboard.pdf** (compilé) :
  1. Volume par **source**
  2. **Top mots-clés** (fréquences)
  3. **Distribution des latences** (box/hist)
  4. **Répartition** statuts HTTP
  5. **Chronologie** (volume/temps)
- Ajouter **1 figure ML** selon la tâche :
  - Classification : matrice de confusion (tableau) ou scores par classe (barres)
  - Clustering : barres “top n-grams par cluster”
  - Reco : tableau “requête → top-k titres”

## Contraintes

- Bibliothèques autorisées  
: `requests`, `json`, `re`, `time`, `logging`, `pandas`, `numpy`, `scikit-learn`, `matplotlib` (*seaborn* optionnel).
- Exécution **automatisée** (pas d’input au runtime).
- Journalisation complète (`logs/marketing.log`).
- Code **modulaire, typé, documenté** (docstrings), 1 responsabilité par module.
- **README.md** (exécution, APIs, choix ML, métriques, limites).

## Plan de réalisation (étapes)

1. **Collecte** (fetcher) : APIs, statuts, latences → `data/raw/`
2. **Nettoyage** (cleaner) : texte propre → `data/processed/clean_data.json`
3. **Analyse** (analyzer) : KPIs (longueur, sources, mots)  
→ `summary.json`, `keywords.csv`
4. **Features** (features) : TF-IDF, sauvegarde `vectorizer.pkl` & `features.npz`
5. **ML** (model / recommender) : entraînement + évaluation (ou reco top-k) → `model.pkl`
6. **Viz** (viz) : générer toutes les figures + `dashboard.pdf`
7. **Rapport final** : mise à jour `summary.json` (inclure métriques ML) + README

## Livrables

- Code : modules + `main.py` (pipeline complet).
- Données : `data/raw/`, `data/processed/`.
- Modèle & vecteur : `data/models/model.pkl`, `vectorizer.pkl`.
- Rapports : `summary.json`, `keywords.csv`, `dashboard.pdf`.
- Figures : `figs/*.png|.svg`.
- Logs : `logs/marketing.log`.
- README.md (APIs, pipeline, choix ML, métriques, reproduction).

Le tout dans un .zip avant le Mercredi 10/12/2025

## Attendus finaux

- Pipeline **fiable** (erreurs gérées, logs complets).
- Données **propres** et **exploitées** (KPIs cohérents).
- ML **justifié**, **mesuré** et **reproductible**.
- Visualisations **lisibles** et **pertinentes**.
- Code **propre**, **modulaire**, **documenté**.